

OMAP5910
Dual-Core Processor
Silicon Errata

SPRZ016A
August 2002 – Revised April 2003



Copyright © 2003, Texas Instruments Incorporated

REVISION HISTORY

This revision history highlights the technical changes made to the SPRZ016 errata to make it an SPRZ016A revision.

Scope: Deleted all CPU advisories and added new advisories.

PAGE(s) NO.	ADDITIONS/CHANGES/DELETIONS
6	Added "Please consult your local sales representative if you need information concerning previous silicon revisions. Issues related to CPU operation are documented in the <i>TMS320C55x DSP CPU Programmer's Reference Supplement</i> (literature number SPRU652)," to Section 1, Introduction.
10	Removed all CPU advisories.
12–17	Added Section 3.2 through Section 3.5.
18	Added MPU_DCACHE_2 advisory.
19	Added Section 4.2.
20	Added SYS_DMA_2 advisory.
22	Added EMIFS_5 advisory.
27–29	Added UART_4 through UART_7 advisories.
30	Added MMC_2 advisory.
33–34	Added I2C_2 through I2C_3 advisories.
38–43	Added Section 6.8 through Section 6.11.
46	Added SYS_4 advisory.

Contents

1	Introduction	6
1.1	Quality and Reliability Conditions	6
	TMX Definition	6
	TMP Definition	6
	TMS Definition	6
1.2	Revision Identification	7
2	Important Notices and Information About OMAP5910	8
2.1	Useful Information Regarding TMS320C55x Assembler Diagnostic Messages	8
2.1.1	ERROR Diagnostics	8
2.1.2	WARNING Diagnostics	8
2.1.3	REMARK Diagnostics	8
3	DSP Subsystem Advisories	10
3.1	DSP DMA Advisories	10
	DSP_DMA_1 DSP EMIF/DMA Port Hangs During EMIF Bus Error	10
	DSP_DMA_2 DSP DMA IDLE Prevents Transfer Completion	10
	DSP_DMA_3 Potential Deadlock in Burst Accesses	11
3.2	DSP ICACHE Advisories	12
	DSP_ICACHE_1 DSP Instruction Cache Two-Way Set-Associative Line Valid Bit Set Incorrectly	12
3.3	DSP Emulation Advisories	13
	DSP_EMU_1 Hardware Breakpoint Set on the Instruction Immediately Following a Conditional Instruction Fails to Halt CPU	13
	DSP_EMU_2 DSP IDLE Interrupt Not Caught When Emulator is Connected	14
3.4	DSP Hardware Accelerator Advisories	15
	DSP_HWA_1 Pixel Interpolation Hardware Accelerator	15
3.5	DSP EMIF Advisories	17
	DSP_EMIF_1 DSP IDLE Wakeup With Program Code in External Memory and EMIF Domain in IDLE Hangs	17
4	MPU Subsystem Advisories	18
4.1	MPU Data-Cache Advisories	18
	MPU_DCACHE_1 Data Cache Transparent Mode Restriction During Copy-Back Operation	18
	MPU_DCACHE_2 Data Cache Entry Operations With VA Depends On CleanCache Mode	18
4.2	MPU DMA Advisories	19
	MPU_DMA_3 DMA Hardware Synchronized Channel is Not Disabled for TIPB Access When DMA's Root Clock is Cut Off	19

4.3	System DMA Advisories	20
	SYS_DMA_1 DMA Clocks Turned Off During Transfers Allows Corruption	20
	SYS_DMA_2 Pending System DMA Request Causes Erroneous Transfer	20
5	Traffic Controller Subsystem Advisories	21
5.1	EMIF Slow (EMIFS) Advisories	21
	EMIFS_1 Burst Writes in EMIFS Causes Latency of Two TC Clock Cycles Extra From Second Data Write in the Data Path	21
	EMIFS_2 WELEN = 0 and FDIV = 1 With 16-Bit Memory	21
	EMIFS_3 EMIFS Wait States	22
	EMIFS_4 FLASH.RDY Does Not Operate Correctly	22
5.2	EMIF Fast (EMIFF) Advisories	23
	EMIFF_1 EMIFF Configuration Preventing Deep Sleep Entry	23
6	OMAP5910 Peripheral Advisories	25
6.1	LCD Advisories	25
	LCD_1 Missing the Palette Loading Interrupt	25
6.2	UART Advisories	26
	UART_1 Software Flow Control Mode of UART1/2/3	26
	UART_2 UART Clock Request Prevents Deep Sleep	26
	UART_3 OSC_12M_SEL and EBLR Registers are not Readable	27
	UART_4 UART1, 2, 3 RX Time-Out IRQ With No Status	27
	UART_5 Incorrect Behavior of UART3 TX_EMPTY_CTL_IT Bit in SIR Mode	28
	UART_6 UART3 SIR_LSR[5] (Bit RX_LAST_BYTE) Not Cleared Upon Read in the SIR_LSR Register	29
	UART_7 UART1/2/3 Sleep Mode	29
6.3	MMC/SD Advisories	30
	MMC_1 MMC/SD Does Not Support Stream Mode Reads	30
	MMC_2 Stop Transmissions Command Cannot be Sent During Multi-Block Transfer of the MMC/SD Module	30
6.4	Microwire Advisories	31
	UWIRE_1 Pulldown On the UWIRE.SDI Pin Needs to be Disabled by Software	31
	UWIRE_2 Microwire Interface RX Data Failures Possible	31
6.5	I²C Advisories	32
	I2C_1 I ² C Prescaler Value of 0 Not Supported in Slave Mode	32
	I2C_2 Transfer Using Polling of I ² C XRDY Bit Fails	33
	I2C_3 Maximum SCL Frequency Configuration in I ² C Master Mode With PSC = 1	34

6.6	USB Function Advisories	35
USBF_1	Read of USB Function Data Register Has a Side-Effect and Should Not be Read From Emulator	35
USBF_2	USB Function Suspend Functionality in HMC_MODE 13 and HMC_MODE 15	36
USBF_3	USB Function Double-Buffering Not Supported	36
6.7	USB Host Advisories	37
USBH_1	Remote Wake Non-Functional Through TLL in HMC_Mode Settings 9, 10, 11, 12, 14, 21, 23, 24, and 25	37
6.8	32K Timer Advisories	38
TIMER_1	32K Timer Erroneous Interrupt	38
TIMER_2	Timer32K Reload TRB Bit Does Not Work Correctly	39
6.9	MPUIO Advisories	40
MPUIO_1	Latch of MPUIO INPUT_LATCH Register is Disabled During TIPB Read Access	40
MPUIO_2	MPUIO GPIO_INT is no Longer Generated	41
6.10	PWT Advisories	42
PWT_1	PWT Signal Can Stop in its High State	42
6.11	Camera Interface Advisories	43
CMR_1	Shutdown of Camera Interface	43
7	OMAP5910 Device/System Level Advisories	44
7.1	System Advisories	44
SYS_1	Timeout Abort on a Posted-Write Access in the TIPB Bridge	44
SYS_2	Write Followed by Immediate Read Not Supported on Specific Addresses (TIPB Switch and PWT Module)	44
SYS_3	Impact on I _{DDC(0)} Current if DSP Held in Reset Without Proper Initialization	45
SYS_4	ESD Susceptibility	46
8	Documentation Support	47

1 Introduction

This document describes the silicon updates to the functional specifications for the OMAP5910, silicon Revisions C and G. Please consult your local sales representative if you need information concerning previous silicon revisions. Issues related to CPU operation are documented in the *TMS320C55x DSP CPU Programmer's Reference Supplement* (literature number SPRU652).

NOTE: Revision G silicon has undergone Electrostatic Discharge testing and the result is a 500-V maximum Human Body Model ESD immunity. For this reason, Revision G is sold under customer waiver acceptance. Design changes have been made to address the issue in the next silicon revision. Test data is pending.

1.1 Quality and Reliability Conditions

TMX Definition

Texas Instruments (TI) does not warranty either (1) electrical performance to specification, or (2) product reliability for products classified as TMX. By definition, the product has not completed data sheet verification or reliability performance qualification according to TI Quality Systems Specifications.

The mere fact that a TMX device was tested over a particular temperature and voltage ranges should not, in any way, be construed as a warranty of performance.

TMP Definition

TI does not warranty product reliability for products classified as TMP. By definition, the product has not completed reliability performance qualification according to TI Quality Systems Specifications; however, products are tested to a published electrical and mechanical specification.

TMS Definition

Fully-qualified production device

1.2 Revision Identification

To designate the stages in the product development cycle, TI assigns prefixes to the part numbers of all OMAP™ processors and support tools. Each commercial OMAP platform member has one of three prefixes: X, P, or null (no prefix). Texas Instruments recommends two of three possible prefix designators for its support tools: TMDX and TMDS. These prefixes represent evolutionary stages of product development from engineering prototypes (TMDX) through fully qualified production devices/tools (TMDS).

Device development evolutionary flow:

- X** Experimental device that is not necessarily representative of the final device's electrical specifications and may not use production assembly flow. (TMX definition)
- P** Prototype device that is not necessarily the final silicon die and may not necessarily meet final electrical specifications. (TMP definition)
- null** Production version of the silicon die that is fully qualified. (TMS definition)

Support tool development evolutionary flow:

TMDX Development support product that has not yet completed Texas Instruments internal qualification testing.

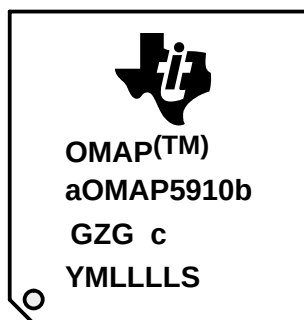
TMDS Fully qualified development support product

TMX and TMP devices and TMDX development-support tools are shipped with appropriate disclaimers describing their limitations and intended uses. Experimental devices (TMX) may not be representative of a final product and Texas Instruments reserves the right to change or discontinue these products without notice.

TMS devices and TMDS development-support tools have been characterized fully, and the quality and reliability of the device have been demonstrated fully. TI's standard warranty applies.

Predictions show that prototype devices (TMX or TMP) have a greater failure rate than the standard production devices. Texas Instruments recommends that these devices not be used in any production system because their expected end-use failure rate still is undefined. Only qualified production devices are to be used.

The device revision can be determined by the symbols marked on the top of the GZG package as shown in Figure 1. Some prototype devices may have markings different from those shown in Figure 1 with the device name in the following format: aOMAP5910bGZGc where a = product level, b = the revision number, and c = an internal TI designator (has no meaning to customer).



NOTE: Qualified devices are marked with no prefix at the beginning of the device name, while nonqualified devices are marked with the letter X at the beginning of the device name.

Figure 1. Example Markings for OMAP5910 GZG Package, Revision C

2 Important Notices and Information About OMAP5910

2.1 Useful Information Regarding TMS320C55x™ Assembler Diagnostic Messages

The TMS320C55x™ (C55x™) DSP assembler will generate three types of diagnostic messages when it detects a potential or probable Silicon Exception.

2.1.1 *ERROR Diagnostics*

The assembler generates ERROR diagnostics in cases where it can fully determine that the code will cause a silicon exception to occur on hardware.

2.1.2 *WARNING Diagnostics*

The assembler generates WARNING diagnostics in cases where it can fully determine that the code will cause a silicon exception to occur on hardware, but which, under certain circumstances, may not be an issue for the user.

2.1.3 *REMARK Diagnostics*

The assembler generates REMARK diagnostics in conditions where it can fully determine that the code may cause a silicon exception to occur on hardware, but the exception itself also depends on non-visible trigger conditions that the assembler has no knowledge of, such as whether interrupts are enabled.

Since the assembler cannot determine the state of these trigger conditions, it cannot know that the exception will affect this code. Therefore, it generates a REMARK to instruct the user to examine the code and evaluate whether this is a potential silicon exception situation. (Please see the following sections for how to suppress remarks in situations where you have determined that the other trigger conditions do not exist.)

Intended Treatment of REMARK Diagnostics

The intent of generating REMARK diagnostics is to inform the user that the code could potentially cause a silicon exception and that it should be reviewed by the user side by side with the trigger conditions and a determination be made whether the code is a potential silicon exception situation.

If the code is determined to be a potential silicon exception situation, users should modify their code to prevent that exception from occurring.

If users determine that their code will not cause a silicon exception based on the trigger conditions, then the REMARK that the assembler generates can be suppressed. There are two methods of doing so; please see the “Suppressing REMARK Diagnostics” section.

Suppressing REMARK Diagnostics

Once the user determines that a silicon exception REMARK diagnostic is not appropriate for the code as written, the REMARK diagnostic can be suppressed in one of the following ways.

- REMARK directives
- REMARK command-line options

TMS320C55x and C55x are trademarks of Texas Instruments.

REMARK Directives:

The `.noremark.remark` directives can be used to suppress the generation of a REMARK diagnostic for particular regions of code. The `.noremark` directive turns off the generation of a particular REMARK diagnostic. The `.remark` directive re-enables the generation of a particular REMARK diagnostic.

A `'noremark ##'` (where `##` is the remark id) directive is placed at the beginning of the region, and a `'remark ##'` directive is placed at the end of the region.

NOTE: The `.noremark.remark` directive combination should always be placed around the entire region of code that participates in the potential silicon exception. Otherwise, spurious diagnostics may still be generated.

Additionally, the user has the option of disabling a silicon exception diagnostic for the entire file by placing just the `.noremark` directive at the top of the assembly file. However, this may be dangerous if, during inevitable code maintenance, the code is modified by someone not familiar with all the exception conditions. Please take great care when using the directives in this manner.

REMARK Command-Line Options:

The compiler shell (cl55) supports a command line option to suppress a particular REMARK diagnostic. The shell option `-ar#` (where `#` is the assembler's silicon exception id as described above) will suppress the named REMARK for the entire scope of all assembly files compiled with that command. Using the option `-ar` without a number will suppress all REMARK diagnostics.

Again, this may be dangerous if, during inevitable code maintenance, the code is modified by someone not familiar with all the silicon exception conditions. Please take great care when using the command-line REMARK options. Using the `.noremark/.remark` directives covering the shortest possible range of source lines is much safer.

3 DSP Subsystem Advisories

3.1 DSP DMA Advisories

Advisory DSP_DMA_1

DSP EMIF/DMA Port Hangs During EMIF Bus Error

Revision(s) Affected: All Revisions

Details: If the EMIF times out on an access, the DSP will get a time-out bus-error interrupt. The time-out condition may also cause a DMA interrupt. In the case where a DMA interrupt occurs, the DMA will not time out, but hangs instead.

Workaround: Whenever an EMIF bus error interrupt occurs the software needs to RESET the DMA and reschedule the transfer.

This exception will not be fixed in future silicon revisions.

Advisory DSP_DMA_2

DSP DMA IDLE Prevents Transfer Completion

Revision(s) Affected: All Revisions

Details: When system modules are placed in IDLE, there is hardware handshaking to ensure IDLE can occur without any system consequences.

The DSP DMA, however, goes into IDLE even though a transfer is occurring. This may prevent an expected from transfer completing.

Workaround: In order to enforce that all DMA transfers are complete before attempting to IDLE the DMA, the DMA status first needs to be checked. The DMA channels then need to be disabled, and the IDLE instruction can then be safely executed.

This exception will not be fixed in future silicon revisions.

**Advisory
DSP_DMA_3***Potential Deadlock in Burst Accesses*

Revision(s) Affected: All Revisions

Details: If a transfer is configured with burst enabled and any of the accessed addresses (notably start address) are not 4x32-bits aligned (i.e., byte address is not multiple of 16) then the DMA may deadlock and the transfer may never terminate.

Workaround: Configure start address, block size, frame size, element size and indexes such that all DMA burst accesses are made on 4x32-bits aligned addresses.

This exception will not be fixed in future silicon revisions.

This same functional limitation is present on the System DMA controller, but the programming restrictions stated in this workaround are documented in the *OMAP5910 Dual-Core Processor Technical Reference Manual* (literature number SPRU602).

3.2 DSP ICACHE Advisories

**Advisory
DSP_ICACHE_1***DSP Instruction Cache Two-Way Set-Associative Line Valid Bit Set Incorrectly*

Revision(s) Affected: All Revisions

Details: Under an unlikely combination of events, there is a possibility for an instruction cache line valid bit to be set incorrectly to valid while the instruction packet (data contained in the line) is not actually correct. Thus, if there is ever a hit to this line, the instruction cache will return an invalid instruction packet and the DSP could try to decode this invalid instruction packet. The following configuration and events are necessary for this issue to occur:

1. DSP instruction cache is configured in 2-way, set-associative mode (note that neither direct mapped or RAMSET modes are affected by this issue).
2. DSP, DSP_MMU, and TC clocks are the same frequency (synchronous with each other).
3. DSP program is in either external SDRAM or internal RAM.
4. The DSP instruction cache has been globally flushed during the invocation of a new program. (The DSP's program is changed and the cache is flushed.)
5. And there is, particular to this bug, a combination of out-of-order DSP accesses to a cache line (speculative branch) and specific timing-related events.

Workaround: Due to the timing nature of the problem, the bug should not occur if the fill rate of the instruction cache is always 1/4th or lower the DSP frequency. It is possible to respect this constraint by observing the following rules:

1. The DSP program is located in either DSP internal SARAM/DARAM or external EMIFF (SDRAM) or EMIFS (FLASH, SRAM) memory space and not the internal 1.5M-bit RAM space.
2. The Traffic Controller and DSP MMU clock rate are one-half or lower the DSP clock rate. That is, the DSP should be running at twice or higher the frequency of the Traffic Controller and DSP MMU. Because the external interfaces are 16 bits wide and the instruction cache fills at 32 bit rate, the 1/4th fill rate constraint will be respected.

A fix for this exception is being considered for future silicon revisions.

3.3 DSP Emulation Advisories

Advisory DSP_EMU_1

Hardware Breakpoint Set on the Instruction Immediately Following a Conditional Instruction Fails to Halt CPU

Revision(s) Affected: All Revisions

Details: A hardware breakpoint set on the instruction immediately following a conditional instruction (conditional branch, conditional return, conditional call) will not halt the CPU when the condition check fails. Application code continues to execute without error, but the user will not see execution halt at the expected location. The source of the hardware breakpoint does not matter. This applies only to hardware breakpoints set on the first instruction after the conditional. Breakpoints set using the Address Control Unit, Data Control Unit, or External Breakpoint Watch Point all fail. Hardware breakpoints work in all other cases. An assembly code example is shown below.

Example 1)

```

    if (cond) call P24
        INST1      << Putting hardware breakpoint here is the problem.
        INST2
        ...

    ...
    if (cond) call P24
        INST1
        INST2      << you can set hardware breakpoint here.
        ...

```

When debugging assembly code in ROM, users must avoid setting breakpoints (ROM breakpoints must be hardware breakpoints) on the instruction following a conditional. There may be some assembly constructs like jump tables that have several conditional calls sequentially in code. Users must be aware that hardware breakpoints should not be used on these type of constructs. An example of several sequential conditional calls is shown below.

Example 2)

```

    ...
    if (cond1) call Func1
    if (cond2) call Func2  << Putting HWBP here is a problem.
    if (cond3) call Func3  << Putting HWBP here is a problem.
    if (cond4) call Func4  << Putting HWBP here is a problem.
    ...

```

When using Code Composer Studio™ Integrated Development Environment (IDE) to debug “C” code in ROM, the debugger automatically sets hardware breakpoints when the user wants to “step over” and “step out” of function calls. The user cannot prevent the breakpoint from being set on the instruction immediately after a conditional. This results in the user seeing the target continue to run when stepping in “C” code. This does not corrupt the target state.

Code Composer Studio is a trademark of Texas Instruments.

Hardware Breakpoint Set on the Instruction Immediately Following a Conditional Instruction Fails to Halt CPU (Continued)

Workaround: Avoid putting a breakpoint at the instruction immediately following the conditional branch/call/return instruction.

If a breakpoint is absolutely needed, then add a NOP between the conditional branch/call/return and the first instruction immediately following it. See the example below.

Example:

```
if (cond) call P24
NOP
INST1          << you can set HWBP here.
INST2
...
```

This exception will not be fixed in future silicon revisions

Advisory DSP_EMU_2

DSP IDLE Interrupt Not Caught When Emulator is Connected

Revision(s) Affected: All Revisions

Details: A DSP IDLE interrupt is not caught when the emulator is connected as shown in the following sequence:

1. Emulator connected
2. RT mode is set
3. Run test code
 - ASM codes sets INTM
 - ASM code clears all pending interrupts in IFR
 - ASM code configures timers, but does not enable
 - ASM code sets ICR to 0x3F
4. Emulator halts CPU
 - PC indicates halted at IDLE instruction
 - DBGSTAT = 0x805F (IDLE, DSUSP, and FXWOK)
 - ISR = 0x3F
 - No interrupts pending in IFR
5. Provide single interrupt.
6. Enable RT int by emulation write to DBIER0
 - Halted background code immediately comes out of idle
 - ISR = 0x3F (interrupt not taken)

Workaround: There is no workaround. The emulator result is different from the functional mode (which will service the ISR). This only affects the debugger mode. Therefore, do not use IDLE when the emulator is connected.

3.4 DSP Hardware Accelerator Advisories

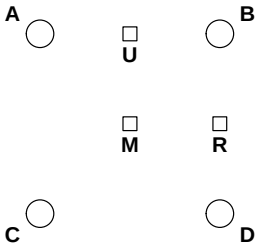
Advisory DSP_HWA_1

Pixel Interpolation Hardware Accelerator

Revision(s) Affected: All Revisions

Details:

The pixel interpolation computation is wrong by one pixel unit in “Decoder” mode, when “Rounding Mode” is set to zero and when half-pixel interpolation is performed in the middle of four full-resolution pixels (i.e., in the middle of two rows of pixels).
In “Decoder” mode, the Pixel Interpolator data path is configured to deliver two interpolated pixels per cycle. Each section of the data path implements the computations shown below, normalized according to video decoding standards:

$$U = \frac{A + B + \text{Rnd}}{2} \quad M = \frac{A + B + C + D + 1 + \text{Rnd}}{4} \quad R = \frac{B + D + \text{Rnd}}{2}$$


U and R results are interpolated from lines and columns, respectively. M is computed from two following full-resolution pixels lines. Two U, R, or M results are computed during each cycle by the data path, according to the instruction being executed by the accelerator for the M results set (the results are denoted as M0 and M1). When “Rounding Mode” (Rnd in the above picture) is set to 1, the faulty data path section is implemented, $M0 = (A+B+C+D+2)/4$, which is the correct result; but when “Rounding Mode” is set to 0, the data path section implements $M0=(A+B+C+D)/4$. This is not correct and generates a bit exactness issue. The M1 result is always correct, regardless of the “Rounding Mode” state.

Workaround:

Do not use the PI data path in the “Decoder” software when computing M type points and when software-rounding is needed. Instead, use a routine consisting of regular C55x instruction combinations. This routine must be called from the point where the one using PI HWA instructions is and should take the same parameters and data organization. Hence, the new routine should perform following steps:

- Unpack the pixels in the CPU
- Diagonal interpolation

Pixel Interpolation Hardware Accelerator (Continued)

The routine below describes the M points computation (diagonal interpolation) for a full block:

Presetting:

- DR1 is set with the block size (8 or 16 pixels)
- AR2 is pointing to the first element in the block

Code example:

Begin:

```

DR2 = DR1 + #2          ; DR2 = blk_size+2
AR4 = AR2               ; AR4 -> block[0][0]
AR5 = AR2
DR3 = DR1 - #1          ; DR3 = block_size - 1
BRC0 = DR3              ; BRC0 = block_size - 1
AR5 = AR5 + DR2         ; AR5 -> block[1][0]
DR0 = #2                ; DR0 = 2
AC3 = DR0 - @rounding_control ; AC3 = 2-rounding_control
|| DR1 = DR1 >> #1      ; DR1 = block_size/2
@rnd_temp = AC3          ; rnd_temp = 2-rounding_control
|| DR1 = DR1 - #1       ; DR1 = block_size/2 - 1
BRC1 = DR1              ; BRC1 = block_size/2 - 1
XAR3 = XDP
AR3 = AR3 + #rnd_temp   ; AR3 -> rnd_temp

|| localrepeat {        ; repeat blk_size times

AC0 = (*AR4+ << #16) + (*AR5+ << #16) ; AC0 = b[i][j]+b[i+1][j]
AC0 = AC0 + (*AR3 << #16) ; AC0 = AC0 + rnd

|| localrepeat {
AC1 = (*AR4- << #16) + (*AR5+ << #16) ; AC0 = b[i][j+1]+b[i+1][j+1]
AC0 = AC0 + AC1 ; AC0 = sum(b[i][j]) + rnd

*(AR4+DR0) = HI(AC0 << #(-2)) ; b[i][j] = (sum(b[i][j])+rnd)/4
|| AC1 = AC1 + (*AR3 << #16) ; AC1 = AC1 + rnd

AC0 = (*AR4- << #16) + (*AR5+ << #16) ; AC0 = b[i][j+2]+b[i+1][j+2]
AC1 = AC1 + AC0 ; AC1 = sum(b[i][j]) + rnd
*(AR4+DR0) = HI(AC1 << #(-2)) ; b[i][j+1] = (sum(b[i][j])+rnd)/4
|| AC0 = AC0 + (*AR3 << #16) ; AC0 = AC0 + rnd
}

mar (*AR4+) || mar (*AR5+)

}

```

A fix is being considered for this exception in future silicon revisions.

3.5 DSP EMIF Advisories

**Advisory
DSP_EMIF_1**

*DSP IDLE Wakeup With Program Code in External Memory and
EMIF Domain in IDLE Hangs*

Revision(s) Affected: All Revisions

Details: The DSP EMIF will not exit IDLE unless it is manually released by setting the corresponding bit in the ICR, then executing the IDLE instruction.

Workaround(s):

1. If the EMIF is IDLEd along with other domains (or by itself), then when an interrupt occurs waking up the other domains, and the CPU continues code execution, the EMIF must be manually removed from IDLE. This is accomplished by writing a 0 to the ICR bits corresponding to all of the domains to be awoken (including the EMIF) and then executing the IDLE instruction.

Furthermore, ensure that the following are executing from internal memory:

- The vector table and the wakeup ISR.
 - If the code is awakened by an unmaskable interrupt, it must get the vector address from the vector table, and vector off to the wakeup ISR.
- IDLE routine
 - In the event that the CPU is awakened by a masked interrupt, the code will continue execution from the point in code that it was IDLEd.

2. Do not IDLE the EMIF

4 MPU Subsystem Advisories

4.1 MPU Data-Cache Advisories

Advisory MPU_DCACHE_1

Data Cache Transparent Mode Restriction During Copy-Back Operation

Revision(s) Affected: All Revisions

Details: The data cache copy-back mode can only be used when the Transparent bit is disabled (set to '0'). When this bit is set to '1', an interleaving of writes and reads will occur during a cache line replacement – write one word of the cache line, read the word that will replace it; write the next word of the cache line, read the word that will replace it; etc. Setting the transparent bit to '0' will cause the writing of the dirty cache line to occur before the new cache line has been read to the cache.

This bit is located in the TI925 Configuration Register in the CP15 coprocessor address space.

Workaround: Do not enable Transparent Mode in the ARM925T Configuration register (it is disabled by default).

This exception will not be fixed in future silicon revisions.

Advisory MPU_DCACHE_2

Data Cache Entry Operations With VA Depends On CleanCache Mode

Revision(s) Affected: All Revisions

Details: The ARM925T data cache is 2-way set-associative. When any of the following cache operations are performed:

Data Cache Flush Entry (MCR p15, 0, Rd, c7, c6, 1)
Data Cache Clean and Flush Entry (MCR p15, 0, Rd, c7, c14, 1)
Data Cache Clean (MCR p15, 0, Rd, c7, c10, 2)

it is possible for the operation to occur on only one of the two cache sets.

Workaround: This problem can be completely avoided by ensuring that bit 2 of the CP15 register (TI925T Configuration) is set to 0 before performing the above operations. The TI925T Configuration register is read using the CP15 operation:

MRC p15, 0, Rd, c15, c1, 0

When zero, bit 2 (the 'C' bit) causes the above operations to simultaneously act on both data cache sets. This ensures that the cache flush or clean operation will be successful. The default state of this bit is zero.

This exception will not be fixed in future silicon revisions.

4.2 MPU DMA Advisories

Advisory MPU_DMA_3

DMA Hardware Synchronized Channel is Not Disabled for TIPB Access When DMA's Root Clock is Cut Off

Revision(s) Affected: All Revisions

Details: If the System DMA's root clock is turned off (by setting ARM_IDLECT2<DMACK_REQ>='0'), and the channel's DMA_CCR register is written to disable the channel, the channel will NOT get disabled until the System DMA clock has been restarted and has been running for a minimum of 2 clock cycles. Therefore, when the next DMA HW request occurs, it is possible for the previous transfer to take place instead of a new transfer.

Workaround: Software must keep the DMA root clock running at all times by setting ARM_IDLECT2<DMACK_REQ>='0'.

Always set ARM_IDLECT2(DMACK_REQ) bit to '0'.

Since the DMA itself has internal automatic clock gating that turns OFF clocks by itself, whenever there are no active DMA requests (assuming internal auto_clock_gating is ON), there is no benefit to software manipulating the DMACK_REQ bit. Automatic clock gating is controlled by bit 3 in the DMA Global Control Register (DMA_GCR<autogating_on>). By default, this bit is set to '1', which enables the DMA clock autogating feature.

There is no impact to power or performance with this workaround.

This workaround ensures that the HW DMA request will occur when DMA clock is enabled, and the transfer will work as expected.

This exception will not be fixed in future silicon revisions.

4.3 System DMA Advisories

**Advisory
SYS_DMA_1***DMA Clocks Turned Off During Transfers Allows Corruption*

Revision(s) Affected: All Revisions

Details: When bit 8 of the ARM_IDLECT2 register (DMACK_REQ) is set to "1", the root clock to the System DMA is turned OFF if there is no valid DMA request. However, it takes about 25 cycles to turn OFF the DMA clocks. During these 25 clock cycles, if a new DMA request is activated, then the clocks to DMA may be turned OFF in the middle of the new DMA transfer, which could lead to a corruption of data.

Workaround: Always set ARM_IDLECT2(DMACK_REQ) bit to "0". Since the DMA itself has internal auto clock gating which will turn OFF clocks by itself when there is no active DMA request (assuming AUTOGATING_ON=1 in the DMA_GCR register, which is the only supported setting of this bit) there is no benefit to software setting the DMACK_REQ bit to "1". There is no impact to power or performance with this workaround.

This exception will not be corrected in future silicon revisions.

**Advisory
SYS_DMA_2***Pending System DMA Request Causes Erroneous Transfer*

Revision(s) Affected: All Revisions

Details: If either the DMA channel is disabled during transfer or a peripheral makes a DMA request after the DMA has completed a transfer, there will be a pending DMA request. This pending DMA request will cause the DMA to initiate a transfer as soon as the channel is enabled again and could create a problem if the associated peripheral is not ready.

Workaround: This issue can be worked around as follows:

1. Enable the DMA channel
2. Flush any pending DMA request on this channel
3. Configure the peripheral

This exception will not be corrected in future silicon revisions.

5 Traffic Controller Subsystem Advisories

5.1 EMIF Slow (EMIFS) Advisories

Advisory EMIFS_1

Burst Writes in EMIFS Causes Latency of Two TC Clock Cycles Extra From Second Data Write in the Data Path

Revision(s) Affected: All Revisions

Details: EMIFS handles burst writes by splitting the transactions into 4 (if the memory connected is 32-bit width) or 8 (if the memory connected is 16-bit width). During split transfers, EMIFS has to follow the single write specifications. But from data write onwards, the data comes 2 TC clock cycles later than the expected. This problem exists irrespective of the initiator (ARM, DSP, DMA, or LB).

Workaround: Increase the WELEN bit field of the EMIFS Chip Select Configuration register to compensate the extra 2 TC clock cycle latency in the data path. The following table shows the requirement for increment of WELEN in case of burst write operation.

FCLKDIV	WELEN (new)	Flash clock
"00"	X + 2	TC_clock/1
"01"	X + 1	TC_clock/2
"10"	X + 1	TC_clock/4
"11"	X + 1	TC_clock/6

This exception will not be fixed in future silicon revisions.

Advisory EMIFS_2

WELEN = 0 and FDIV = 1 With 16-Bit Memory

Revision(s) Affected: All Revisions

Details: When performing an asynchronous operation to 16-bit memories, connected to EMIFS interface, with WELEN=0 and FCLKDIV=00 (configured in EMIFS chip select configuration register), EMIFS generates an extra ready for each access to the host. This can cause the host to interpret the extra ready as the ready for the next access, which can cause the system to hang.

Workaround: Use one of the following two workarounds:

- Program WELEN (of EMIFS configuration registers) greater than 0
- Use FCLKDIV (of EMIFS configuration registers) greater than "00"

This exception will not be fixed in future silicon revisions.

**Advisory
EMIFS_3***EMIFS Wait States*

Revision(s) Affected: All Revisions

Details: Data pipeline in EMIFS is broken during burst write with certain configuration
(WRWST < 3 and FCLKDIV = 0).

Workaround: The EMIFS must be setup with a minimum of 3 write wait states (WRWST > 2) for proper operation if FCLKDIV = 0. For most devices used with EMIFS, this is not a performance constraint since the memories available are slower than this speed.
This exception will not be fixed in future silicon revisions.

**Advisory
EMIFS_4***FLASH.RDY Does Not Operate Correctly*

Revision(s) Affected: All Revisions

Details: FLASH.RDY is improperly synchronized to the internal FLASH clock. Metastability issues can occur.

Workaround: None.
Do not use FLASH.RDY and tie the pin high with an external pullup. Increase the wait state setting for external memory spaces connected to slow external devices through the wait state bits in the EMIF Slow Chip-Select Configuration Registers (EMIFS_CSx_CONFIG, x=0–3).
No corrections are planned for future revisions.

5.2 EMIF Fast (EMIFF) Advisories

Advisory EMIFF_1

EMIFF Configuration Preventing Deep Sleep Entry

Revision(s) Affected: All Revisions

Details: Due to the implementation of the SDRAM power down feature in the EMIFF, it is possible to have a sequence of events where the SDRAM clock will not properly be disabled making it impossible to enter deep sleep.

The following table details every combination of relevant events which lead to deep sleep entry. The sequence of events is from left to right in the table, i.e., the register bit specified in column one is step1 followed by the register bit in step2, etc. Also, all register bits are assumed to be low before the sequence is started. The "Deep Sleep?" column indicates whether the device will successfully enter deep sleep mode for the corresponding sequence (OK) or whether it will fail to enter deep sleep (NOT OK).

STEP1	STEP2	STEP3	STEP4	DEEP SLEEP?
EMIFS_PDE	EMIFF_CLK	EMIFF_PWD	SLFR	OK
EMIFS_PDE	EMIFF_CLK	SLFR	EMIFF_PWD	OK
EMIFS_PDE	EMIFF_PWD	EMIFF_CLK	SLFR	OK
EMIFS_PDE	EMIFF_PWD	SLFR	EMIFF_CLK	OK
EMIFS_PDE	SLFR	EMIFF_PWD	EMIFF_CLK	NOT OK
EMIFS_PDE	SLFR	EMIFF_CLK	EMIFF_PWD	NOT OK
EMIFF_CLK	EMIFS_PDE	EMIFF_PWD	SLFR	OK
EMIFF_CLK	EMIFS_PDE	SLFR	EMIFF_PWD	OK
EMIFF_CLK	EMIFF_PWD	EMIFS_PDE	SLFR	OK
EMIFF_CLK	EMIFF_PWD	SLFR	EMIFS_PDE	OK
EMIFF_CLK	SLFR	EMIFF_PWD	EMIFS_PDE	OK
EMIFF_CLK	SLFR	EMIFS_PDE	EMIFF_PWD	OK
EMIFF_PWD	EMIFS_PDE	EMIFF_CLK	SLFR	OK
EMIFF_PWD	EMIFS_PDE	SLFR	EMIFF_CLK	OK
EMIFF_PWD	EMIFF_CLK	EMIFS_PDE	SLFR	OK
EMIFF_PWD	EMIFF_CLK	SLFR	EMIFS_PDE	OK
EMIFF_PWD	SLFR	EMIFF_CLK	EMIFS_PDE	OK
EMIFF_PWD	SLFR	EMIFS_PDE	EMIFF_CLK	NOT OK
SLFR	EMIFS_PDE	EMIFF_CLK	EMIFF_PWD	NOT OK
SLFR	EMIFS_PDE	EMIFF_PWD	EMIFF_CLK	NOT OK
SLFR	EMIFF_CLK	EMIFS_PDE	EMIFF_PWD	OK
SLFR	EMIFF_CLK	EMIFF_PWD	EMIFS_PDE	OK
SLFR	EMIFF_PWD	EMIFF_CLK	EMIFS_PDE	OK
SLFR	EMIFF_PWD	EMIFS_PDE	EMIFF_CLK	NOT OK

*EMIFF Configuration Preventing Deep Sleep Entry (Continued)***Important notes:**

- The above table was generated with
 - RFRSH_STBY of the EMIFF_SDRAM_CONFIG_2 (bit 0) register = 1, and,
 - PWD_EN of EMIFS_CONFIG_REG (bit 2) register = 1
- EMIFS_PDE = Global Power Down Enable, EMIFS_CONFIG_REG(3)
- EMIFF_CLK = EMIFF sdram clock control, EMIFF_SDRAM_CONFIG(27).
- EMIFF_PWD = EMIFF Power Down Enable, EMIFF_SDRAM_CONFIG(26).
- SLFR = EMIFF Self Refresh Control, EMIFF_SDRAM_CONFIG(0).

Workaround:

In order to ensure that the deep sleep state is entered correctly and the clocks are turned off, it is necessary to make sure that the Global Power Down Enable [EMIFS_CONFIG_REG(3)] is set as the last event of the TC Idle entry procedure and it has to be toggled from 0 → 1.

The recommended sequence is:

1. Set EMIFS_PDE = 0
2. Set EMIFF_CLK, SLFR, EMIFF_PWD to 1 (in any order, or all at once)
3. Make sure that IMIF_PWD bit is set to 1.
4. Set EMIFS_PDE = 1

As there is a simple software workaround, this exception will not be fixed in future silicon revisions.

6 OMAP5910 Peripheral Advisories

6.1 LCD Advisories

Advisory LCD_1	Missing the Palette Loading Interrupt
-------------------	---------------------------------------

Revision(s) Affected:	All Revisions
Details:	When LCD is in Palette and Data loading mode and the palette loading interrupt occurs, the LCD sends an interrupt, but the status register bit does not get set. ARM will see the interrupt, but will not be able to identify what caused it because LCD status register does not capture it. The real intention of this bit, however, is for palette loading mode only. If the user is in palette loading mode only, then the operation is correct; the interrupt is sent and the status bits are properly set.
Workaround:	Do not use this interrupt when operating in Palette and Data loading mode. If operating in this mode then this interrupt should be masked. This exception will not be fixed in future silicon revisions.

6.2 UART Advisories

Advisory UART_1

Software Flow Control Mode of UART1/2/3

Revision(s) Affected: All Revisions

Details: When software flow control mode is enabled, the UART (MODEM or IrDA) will compare incoming data with XOFF1 (and/or XOFF2) programmed characters or XON1/2:

- If a correct XOFF is received, the transmission is halted.
- If a correct XON is received the transmission is restarted.
- If XOFF1 and XOFF2 are used they must be received sequentially in order to halt the transmission.

If parity, framing or break error occurs when XOFF is received, the transmission should not be stopped but an error should be reported.

There are two issues:

- XOFF with framing error (only this one) stops the transmission.
- With XOFF1 + word with Parity or break error + XOFF2, the transmission is stopped. It is not correct because XOFF2 does not follow immediately XOFF1. (Note: the sequence XOFF1 + word without error + XOFF2 does not stop the transmission)

Workaround: Use one of the following two workarounds:

- use hardware flow control.
- use software to perform software flow control without hardware assist.

This exception will not be fixed in future silicon revisions.

Advisory UART_2

UART Clock Request Prevents Deep Sleep

Revision(s) Affected: Revision C

Details: OMAP5910 was intended to have a feature that if communication is sent on the UART2.RX signal while the device is in deep sleep mode then the device will automatically wake up from deep sleep. This function does not work properly and consequently, if UART2.RX activity occurs while the device is awake, this will incorrectly create a pending wakeup request from UART2 preventing the device from entering deep sleep mode even when all other conditions required for deep sleep are met.

Workaround: To allow deep sleep mode is to be used, it is recommended that UART2 be used for data transmission only and that a different UART (UART1 or UART3) be used to receive serial data. Note that this issue in no way impacts other deep sleep operation and is limited to the constraint outlined in this notice.

A fix for this exception is being considered for future silicon revisions.

Advisory
UART_3

OSC_12M_SEL and EBLR Registers are not Readable

Revision(s) Affected:

All Revisions

Details:

Several UART configuration registers are write-only and cannot be read by the MPU or the DSP. These registers are: the OSC_12M_SEL register of UART1, 2, and 3 and EBLR register of UART3.

Workaround:

Writes to these registers operate correctly but software will not be able to confirm written value with a read. Write errors to the OSC_12M_SEL registers will exhibit themselves as baud-rate inaccuracies.

This exception will not be fixed in future silicon revisions.

Advisory
UART_4

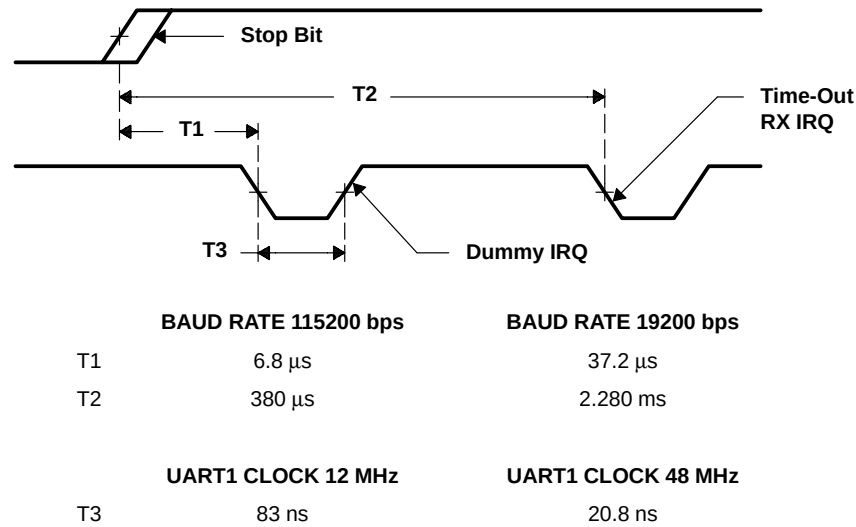
UART1, 2, 3 RX Time-Out IRQ With No Status

Revision(s) Affected:

All Revisions

Details:

The UART1, 2, 3 modules generate an interrupt without corresponding status when a receive time-out occurs. This time-out is generated when received data is present in the FIFO and is not read (MPU or DMA) during a defined time.



Workaround:

In the ISR, check bit 0 of the IIR register, which signals that no interrupt has been generated by the UART peripheral; consequently, no associated action is necessary. However, a second receive time-out interrupt occurs which does show the correct status of time-out RX IRQ and thus can be treated.

This exception will not be fixed in future silicon revisions.

**Advisory
UART_5****Incorrect Behavior of UART3 TX_EMPTY_CTL_IT Bit in SIR Mode**

Revision(s) Affected: All Revisions

Details: The SCR(3) bit (TX_EMPTY_CTL_IT) is non-functional and cannot be used to generate an interrupt when the TX FIFO and TX shift register are empty. The status bit LSR(7) (THR_EMPTY) signals only that the TX FIFO is empty and not that the TX shift register is also empty.

With the UART3 in IRDA mode, there is no easy way by hardware to know when the last character has been transmitted; after the FIFO is empty, there will still be some bits to transmit (the last character, the EOF, and 2 bytes of CRC16). This can create a problem in the following situations:

1. Before setting the IDLE mode, the user has to know when all the characters have been transmitted.
2. Enabling the receiver: When there is an echo signal on the receiver due to the transmitter.
3. Programming a new transmit frame: the transmitted frame is unexpected.

Workaround: SW can set a timer to track the minimum amount of time needed to send one character.

$T_{wait_min} = \text{character_length} * \text{Baud clock period}$
character_length: 5, 6, 7, or 8 bits

Then when the TX FIFO is empty and gives an interrupt, the SW needs to wait an amount of time to ensure that the contents of the TX shift register have been emptied before initializing a new operation. The minimum delay is equal to the duration of a 56-bit transmission. The following is an example using a data format of 8-bit and 2 stop bits (worst case):

$T_{wait_min} = 56 / \text{baud rate}$
At 9600 bps, a delay of 5.833 ms is necessary.
At 115200 bps, a delay of 486 μ s is necessary.

This exception will not be fixed in future silicon revisions.

**Advisory
UART_6**

UART3 SIR_LSR[5] (Bit RX_LAST_BYTE) Not Cleared Upon Read in the SIR_LSR Register

Revision(s) Affected: All Revisions**Details:** Bit 5 (RX_LAST_BYTE) of the SIR_LSR register signals that the last byte of the frame has been received. Contrary to what it was specified in the OMAP5910 TRM documentation, it is not automatically cleared when the corresponding SIR_LSR register is read.**Workaround:** In order to reset the "RX_LAST_BYTE" bit of the SIR_LSR register, the following steps have to be taken:

1. First read the last received byte.
2. Then read the SIR_LSR Register.

This exception will not be fixed in future silicon revisions.

**Advisory
UART_7**

UART1/2/3 Sleep Mode

Revision(s) Affected: All Revisions**Details:** When UARTx is set in the sleep mode, and if a new data is received, this data gets shifted and consequently is corrupted.

When sleep mode is set, the following occur:

- bit 4 of the IER register for UART1/2
- bit 4 of the UART_IER register for UART3 in UART mode
- bit 3 of the MDR1 register for UART3 in SIR mode

UARTx enters in sleep mode (that is, the module clock and the baud clock are switched off) after some amount of time, which corresponds to the end of the current activity.

A counter (idle_cnt), which is working with the baud clock, is used to wait for this amount of time. The limit value of this counter depends on the UART configuration:

- In UART mode: limit value = $[4 * (\text{word_length}) + 12] * 16$. Where the word_length = 1 start bit + number of bits in the word + number of stop bit(s) + parity bit (if applicable).
- In SIR mode: limit_value = 52

If new data occurs on the RX line in the same cycle (or one cycle before) that the counter reaches this limit value, the activity on the RX line is not taken into account and the module and baud clock are switched off. Consequently, the start bit is missed and the received data will be shifted.

Note: This problem could only occur if the sleep mode is set.

Workaround: There is no workaround to use the sleep mode.

6.3 MMC/SD Advisories

**Advisory
MMC_1***MMC/SD Does Not Support Stream Mode Reads*

Revision(s) Affected: All Revisions

Details: The MMC/SD controller does not support stream mode read operations with MMC devices (SD cards do not support stream mode.) The command associated with stream mode reads is:

CMD11 – READ_DAT_UNTIL_STOP

Workaround: Block oriented read commands (class 2) must be used for read operations.
This exception will not be fixed in future silicon revisions.

**Advisory
MMC_2***Stop Transmissions Command Cannot be Sent During Multi-Block Transfer of the
MMC/SD Module*

Revision(s) Affected: All Revisions

Details: If the Stop_Transmissions command (CMD12) is sent during a multiple block data transfer (read/write), the occurrence of interrupt request Block_RS at the end of the data transfer can cause the MMC/SD controller to hang.

Thus, there is a limitation that during multiple block data transfer the stop command must not be sent before all of the blocks programmed in MMC_NBLK register have been sent/received.

Workaround: Software needs to send the CMD12-Stop_Transmission only after the interrupt for the Block_RS has been received at the end of the multiple block data transfer (read/write).
This exception will not be fixed in future silicon revisions

6.4 Microwire Advisories

Advisory UWIRE_1

Pulldown On the UWIRE.SDI Pin Needs to be Disabled by Software

Revision(s) Affected: All Revisions

Details: On the UWIRE.SDI pin (ball J14 of the 289-pin BGA device when configured for UWIRE) pin, there is a pulldown active by default. The inactive level of the signal is high, so the pulldown needs to be disabled in software in OMAP5910 native mode otherwise power is wasted. The pulldown is disabled in compatibility mode.

Workaround: Write 0x0000EAEFh to the COMP_MODE_CTRL_0 register (base address: FFFE:1000 + offset 0x0C) to put the part in OMAP5910 native mode.

Before putting the device into OMAP5910 native mode, set CONF_PDEN_WIRE_SDI_R (bit 18) to 1 of the PULL_DWN_CTRL_1 register (MPU byte address FFFE:1044).

This exception will not be fixed in future silicon revisions.

Advisory UWIRE_2

Microwire Interface RX Data Failures Possible

Revision(s) Affected: Revision C

Details: There are RX data failures which occur when specific configurations are used:

Configuration #1:

CSx_EDGE_RD=0
CSx_EDGE_WR=1
Delay of 1.5 SCLK cycles between last bit transmitted and first receive bit captured.

Configuration #2:

CSx_EDGE_RD=1
CSx_EDGE_WR=0
Delay of 2.5 SCLK cycles between last bit transmitted and first receive bit captured.

In Configuration #1, an extra bit of receive data is captured in the RX register. In Configuration #2, the first bit of RX data will not be captured.

Workaround: Configuration #1: In this configuration, bit 0 can be ignored by right shifting the RX register value by 1 after reading the data from the RX register. This workaround cannot be used in the 16-bit mode since 1 bit of data is lost due to the extra captured bit. In 16-bit mode, a different configuration must be used.

Configuration #2: No workaround exists. A different configuration must be used.

6.5 I²C Advisories

Advisory I2C_1

I²C Prescalar Value of 0 Not Supported in Slave Mode

Revision(s) Affected: All Revisions

Details: The I²C peripheral does not detect a start condition on the I²C Bus™ when the prescalar value is programmed to a divide by 1 value. When the I2C_PSC register stays at its default value of 0 (divide by 1 of the system 12-MHz clock), the Bus Busy (bit 12) bit of the I2C_STAT is not set. Additionally in this case, the I²C peripheral does not recognize the address sent by the master.

Register addresses affected are:

I2C_PSC register = byte address FFFB:380C

BB bit (bit 12) of I2C_STAT register = byte address FFFB:3802

Workaround: If operating in Slave Mode set PSC greater than or equal to 1. If operating in Master Mode above 100 kbps then set PSC to 0.

As the System Clock is a 12-MHz clock (83.3 ns), the internal sampling clock will have a period of 166.6 ns when PSC is set to 1. The I²C performance and functionality is still in conformance to the I²C specification, however, a pulse width less than 166.6 ns (which includes 83.3 ns pulses) will be filtered.

This exception will not be fixed in future silicon revisions.

**Advisory
I2C_2****Transfer Using Polling of I²C XRDY Bit Fails**

Revision(s) Affected: All Revisions

Details: The resynchronization between the MPU and the I²C by polling the XRDY bit (bit 4 of the I2C_STAT register) for a data transmission does not work. During transmission, there is a possibility that XRDY is high (I2C_DATA is ready to receive a new data word of two bytes) while there is only one byte empty in the transmitter FIFO. Thus, when the MPU writes two bytes, then one of the bytes in the FIFO is overwritten and lost.

It should be noted, however, that this bug does not affect the transmission when using interrupt mode. A transmit data interrupt (INTCODE = 101b) only occurs when there are two bytes available for the MPU to write into the transmit FIFO. Thus, there is no possibility for the MPU to overwrite data in the transmit FIFO on reception of a transmit data interrupt.

Workaround: There are two possibilities for workaround:

1. Use interrupt mode transmit data interrupt (INTCODE = 101b).
2. If polling mode is mandatory, then data transmission using polling can be accomplished by the use of the XUDF bit (bit 10 of the I2C_STAT register).

The XUDF bit signals that the transmitter has experienced underflow:

- In master transmit mode, underflow occurs when the transmit shift register (ICXSR) is empty, the transmit FIFO is empty, and there are still bytes to transmit (DCOUNT not equal to 0).
- In the slave transmit mode, underflow occurs when the transmit shift register (ICXSR) is empty, the transmit FIFO is empty, and there are still bytes to transmit (read request from external I²C master).

This exception will not be fixed in future silicon revisions.

**Advisory
I2C_3**Maximum SCL Frequency Configuration in I²C Master Mode With PSC = 1

Revision(s) Affected: All Revisions

Details: When the I²C is configured in master mode and the PSC is set to 1 (I²C functional clock internally divided by 2), the SCL clock frequency is limited to a minimum setting of 3 (SCLL and SCLH registers setting to 3). This configuration corresponds to a maximum SCL frequency while PSC is set to 1.

Note: The slave mode and the configuration of PSC='0' in master mode are not affected by this issue.

Workaround: In master mode, set the PSC to 0.

However, as PSC = 0 does not work in slave mode, the switch from slave to master or master to slave will require a PSC change. The PSC change requires the following sequence:

- Disable I²C
- Change PSC
- Enable I²C

Note: When the I²C is disabled, it is not possible to observe the status of the I²C system bus. This exception will not be fixed in future silicon revisions.

6.6 USB Function Advisories

Advisory USBF_1

Read of USB Function Data Register Has a Side-Effect and Should Not be Read From Emulator

Revision(s) Affected: All Revisions

Details: Emulation/debug read access (ARM TIPB Suspend active) for USB Function DATA register works only if the end point (the last offset address) is selected. If endpoint is not selected, there is no response from USB Function, and the ARM processor hangs.

Emulation/debug read access (ARM TIPB Suspend active) for USB Function DATA_DMA register works only when the DMA Request is active. If DMA request is not active, there is no response from USB Function, and the system DMA Controller hangs.

The registers affected are:

DATA = byte address FFFB:4008

DATA_DMA = byte address FFFB:4048

No other registers in the USB Function are affected.

Workaround: Do not read this register from emulator. The Code Composer Studio memory map may be configured so that memory windows will not cause accesses to these registers. Either of these two methods can be used.

Option 1 – use Code Composer Studio "Options" pull down menu's "Memory Map" window to define the two registers as "protected". This sequence will need to be performed every time you open Code Composer Studio. Perform the following steps:

Select "Memory Map" on the "Options" menu
 Set the Starting Address field to 0xFFFB4008.
 Set the Length field to 4.
 Set the Attribute field to None – No Memory/Protected.
 Set the Access Size field to Auto.
 Make sure the Memory Mapping box is checked.
 Click the Add Button
 Set the Starting Address field to 0xFFFB4048.
 Set the Length field to 4.
 Set the Attribute field to None – No Memory/Protected.
 Set the Access Size field to Auto.
 Make sure the Memory Mapping box is checked.
 Click the Add Button.
 The Memory Map List window should now include the following lines:
 0xFFFB4008-0xFFFB400B: NONE
 0xFFFB4048-0xFFFB404B: NONE
 Click the OK Button. See the Code Composer Studio on-line Help for further details on Memory Mapping.

Read of USB Function Data Register Has a Side-Effect and Should Not be Read From Emulator (Continued)

Option 2 – Modify the Code Composer Studio GEL file that is automatically loaded at Code Composer Studio boot time so that it preinitializes the memory map so that these registers are protected. Add the following lines to the GEL file:

```
GEL_MapAddStr(0xFFFFB4000, 0, 0x000000008, "R|W|AS2", 0);
GEL_MapAddStr(0xFFFFB4008, 0, 0x000000004, "NONE", 0);
GEL_MapAddStr(0xFFFFB400C, 0, 0x00000003C, "R|W|AS2", 0);
GEL_MapAddStr(0xFFFFB4048, 0, 0x000000004, "NONE", 0);
GEL_MapAddStr(0xFFFFB404C, 0, 0x0000007B4, "R|W|AS2", 0);
```

You may need to modify other GEL_MapAddStr or GEL_MapAdd operations in the GEL file that already define this area or overlap this area.

See the Code Composer Studio on-line Help for further details on Memory Mapping, GEL files, and the GEL_MapAddStr and GEL_MapAdd operations.

This exception will not be fixed in future silicon revisions.

Advisory 1 USBF_2

USB Function Suspend Functionality in HMC_MODE 13 and HMC_MODE 15

Revision(s) Affected: All Revisions

Details: When in HMC_MODE 13 or HMC_MODE 15, the TLL receives its Suspend and Pullup Enable signals from the USB Function. In its default operating mode, the USB Function will not transition from Suspend to enabled until it senses that it has successfully enabled the Pullup. The TLL prioritizes its Suspend input over its Pullup Enable input, so the TLL will not signal the presence of the pullup, and the USB Function cannot sense that it has enabled the pullup. The USB Function will not exit Suspend mode.

Workaround: Do not use the Transceiverless Link Logic with the USB Function and an external USB Host Controller. Use a transceiver-based solution instead.

This exception will not be fixed in future silicon revisions.

Advisory USBF_3

USB Function Double-Buffering Not Supported

Revision(s) Affected: All Revisions

Details: USB Function Double Buffering does not function properly.

Workaround: Use Single-buffer mode instead of double buffering mode.

This exception will not be fixed in future silicon revisions.

6.7 USB Host Advisories

Advisory USBH_1

Remote Wake Non-Functional Through TLL in HMC_Mode Settings 9, 10, 11, 12, 14, 21, 23, 24, and 25

Revision(s) Affected: All Revisions

Details: When in HMC_MODE 9, 10, 11, 12, 14, 21, 23, 24, or 25, the USB Host will not receive Remote Wake from the external USB Function Controller connected to the pins associated with the Transceiverless Link Logic. The external USB Function Controller cannot wake the USB link from USB Suspend using Remote Wake.

This does not affect other connectivity which does not use the Transceiverless Link Logic.

Workaround(s):

1. Systems which do not use USB Remote Wake through the Transceiverless Link Logic may ignore this errata.
2. Use a transceiver-based connection (may require use of a different HMC_MODE or a different set of OMAP5910 pins).
3. Use an OMAP5910 GPIO pin and software monitoring to indicate when software should take the appropriate USB Host Controller port out of USB Suspend state.
4. Avoid putting the USB Host Port which uses the Transceiverless Link Logic into USB Suspend.

This exception will not be fixed in future silicon revisions.

6.8 32K Timer Advisories

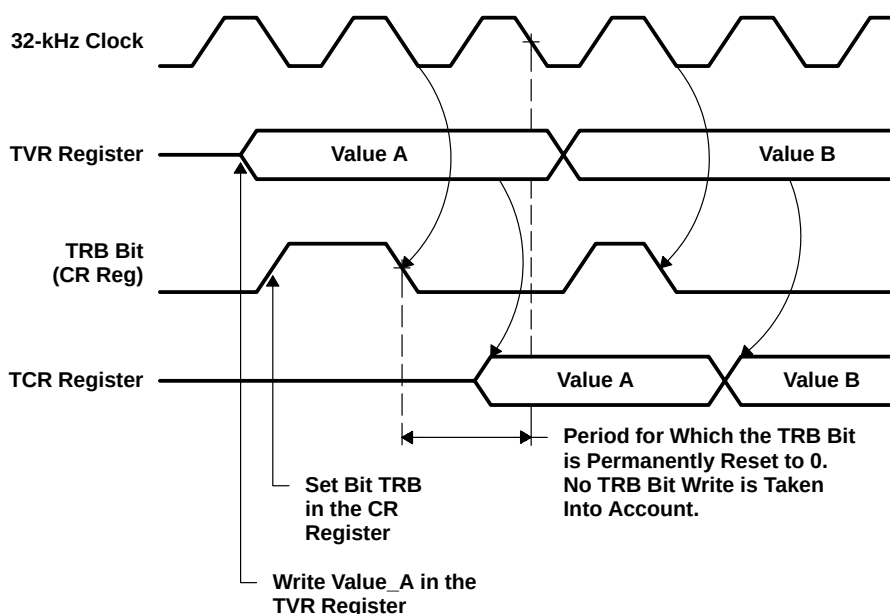
Advisory TIMER_1	32K Timer Erroneous Interrupt
---------------------	-------------------------------

Revision(s) Affected:	All Revisions
Details:	When using the 32K timer with TVR = 1 or 0 (tick value register), an interrupt will be created as soon as the interrupt is unmasked even though the timer32K is not started. Also, the interrupt will stay asserted (low) until the timer is reloaded. This is not an issue as the interrupt should be set up to be edge-sensitive.
Workaround:	Do not use the 32K timer with TVR = 1 or 0. This exception will not be fixed in future silicon revisions.

**Advisory
TIMER_2***Timer32K Reload TRB Bit Does Not Work Correctly***Revision(s) Affected:** All Revisions

Details: The Timer32K Reload bit (TRB) is used to load the TCR register with the value contained in the TVR register. Once the TCR is loaded, the TRB bit should be automatically reset to '0'. The user should be able to use the TRB bit (by polling) to know when to load a new value in the TVR.

However (see the timing diagram below), the TRB bit is reset to zero by the falling edge of the 32-kHz clock only after a maximum of $1.5 \times 32\text{-kHz}$ clock period (or minimum $0.5 \times 32\text{-kHz}$ clock period) after the TRB bit is set. The TCR register is loaded after a maximum of two 32-kHz clock periods (or minimum of $1 \times 32\text{-kHz}$ clock periods) on the rising edge of the 32-kHz clock. After the TRB is reset, there is one 32-kHz clock period where a new TRB setting will not be taken into account by the module. If the user programs the TVR and updates the TRB bit within this 32-kHz clock period, the new TVR will not be loaded correctly into the TCR.



Workaround: To load a new value into the TCR, poll the TRB bit. When the TRB is equal to '0' (signaling that the previous value has been loaded), the user has to wait for an additional one 32-kHz clock period, before loading a new value in the TVR and setting the TRB bit to '1'.

This exception will not be fixed in future silicon revisions.

6.9 MPUIO Advisories

Advisory MPUIO_1

Latch of MPUIO INPUT_LATCH Register is Disabled During TIPB Read Access

Revision(s) Affected: All Revisions

Details:

A continuous polling on the MPUIO INPUT_LATCH register does not work correctly.

The MPUIO_INPUT_LATCH register has been implemented with a LATCH, this LATCH is:

- Enabled when there is no TIPB read access to the MPUIO INPUT_LATCH register. The register is directly updated by the MPUIO inputs.
- Disabled when there is a TIPB read access to the MPUIO INPUT_LATCH register. The register holds its last value until LATCH is enabled.

Furthermore, in the MPUIO module, the TIPB STROBE is not used to validate a TIPB read access. Consequently, as long as an MPUIO INPUT_LATCH register address is present on the TIPB bus, the corresponding MPUIO INPUT_LATCH register LATCH is selected and therefore disabled (does not take anymore into account of the MPUIO inputs).

Workaround:

In order to avoid this during the MPUIO INPUT_LATCH register polling, a dummy TIPB access (the only condition is to do this access to another TIPB address) has to be done just after the MPUIO INPUT_LATCH register read.

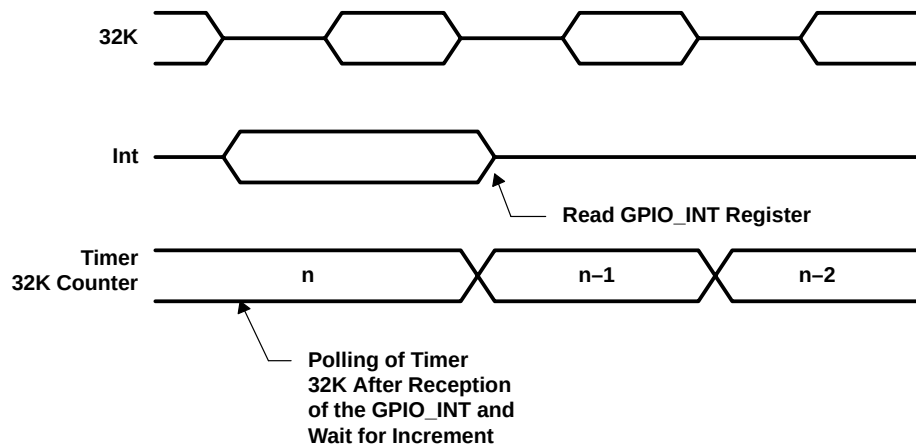
This exception will not be fixed in future silicon revisions.

**Advisory
MPUIO_2***MPUIO GPIO_INT is no Longer Generated***Revision(s) Affected:** All Revisions

Details: The reset of the MPUIO GPIO Interrupt is done asynchronously when a GPIO_INT register read occurs. The release of this reset is done synchronously with the 32-kHz clock to avoid any reset recovery time violation. A read of the GPIO_INT register while Clock 32k is low or transitioning can result in a deadlock of the state machine that causes all further interrupts on the subject MPUIO pin to be ignored.

Workaround: After receiving the MPUIO GPIO_INT, the MPU should observe the 32-kHz system clock and then read the GPIO_INT register only just after a 0-to-1 transition on the 32-kHz system clock. There are different ways to detect a 0-to-1 transition on the 32-kHz system clock:

- Polling the timer 32k counter until there is an increment. See the waveforms below, which describe the procedure.



- Loop-back the CLK32K_OUT output on one available MPUIO, GPIO, or KB.R input, and polling the corresponding register.

Note that the maximum delay inserted by this workaround, before the interrupt clears, is 31 μ s (one 32K clock period).

This exception will in future silicon revisions.

6.10 PWT Advisories

Advisory PWT_1	PWT Signal Can Stop in its High State
-------------------	---------------------------------------

Revision(s) Affected:	All Revisions
Details:	The PWT pin (UART3.TX) will sometimes stop in the high state after it is disabled. The PWT is disabled by: • First, switching off the tone (VCR[15:0]=0x0) • Then shutting off the PWT clock (GCR[15:0]=0x0) In a typical application, the PWT pin is connected to the gate of a transistor, which drives a PIEZO speaker. If the PWT pin stops in its high state after being disabled, it can cause (significant) unwanted current across the speaker and consequently increase system power consumption. The high state on the PWT pin persists when OMAP5910 enters Deep Sleep Mode.
Workaround:	After disabling the PWT, modify the pin multiplexing of this pin to select configuration mode 0, the UART3.TX pin. When UART3 is disabled, a low level is permanently forced on this pin. The pin-muxing is controlled by FUNC_MUX_CTRL_6 bits 2:0 (CONF_TX3_R). This exception will in future silicon revisions.

6.11 Camera Interface Advisories

Advisory CMR_1

Shutdown of Camera Interface

Revision(s) Affected: All Revisions

Details: The camera interface logic can generate CAM.EXCLK frequencies based on either the 12-MHz system clock or by a 48-MHz clock provided by the DPLL in the ULPD peripheral. The 12-MHz clock is enabled by setting bit 5 (MCLK_EN) of the camera interface CTRLCLOCK register. The 48-MHz clock is enabled either by setting bit 6 (DPLL_EN) or by selecting a clock mode where bit 2 is '1' (this is the MSB of the FOSCMODE field).

To enter Deep Sleep mode, it is necessary to eliminate the camera module requests for both the 12-MHz and 48-MHz clocks. To eliminate the 48-MHz clock request, the 12-MHz clock needs to be running (i.e., these two clocks cannot be stopped simultaneously).

Workaround: To completely shut down the camera module, two separate writes to the CTRLCLOCK register are required.

1. The first write must clear DPLL_EN (bit 6 = '0') and select an FOSCMODE where bit 2 is '0'. During this same write, other register bits may also be changed, but MCLK_EN must still be '1'. This keeps the 12-MHz clock running.
2. The second write must clear MCLK_EN (bit 5 = '0').

This workaround is not necessary if neither bit 6 nor bit 2 were enabled. There are no other restrictions on these consecutive writes (i.e., they can be consecutive operations).

This exception will not be fixed in future silicon revisions.

7 OMAP5910 Device/System Level Advisories

7.1 System Advisories

Advisory SYS_1

Timeout Abort on a Posted-Write Access in the TIPB Bridge

Revision(s) Affected: Revision C

Details: An issue has been detected when **posted-write** is enabled in the TIPB Bridge, and a **posted-write** transaction causes a timeout abort. In this case the TIPB Bridge wrongly generates a data strobe to the ARM, which could cause the ARM to hang.

Workaround: Disable the **posted-write** in the both the Public and Private TIPB Bridges. By default (coming out of RESET), the **posted-write** is disabled. The posted-write feature can be disabled by setting `ARM_TIPB_CNTL[1:0] = "00"`.

A fix is being considered for this exception in future silicon revisions.

Advisory SYS_2

Write Followed by Immediate Read Not Supported on Specific Addresses (TIPB Switch and PWT Module)

Revision(s) Affected: All Revisions

Details: A write followed by an immediate read does not work on the ARM address space defined below. If a read occurs immediately after the write to the same address, then the read will return incorrect data (not the data that was just written).

The affected address spaces are:

TIPB Switch module: Address **FFFB:C800 to End Address FFFB:CFFF**

PWT module: Address space **FFFB:6000 to End Address FFFB:67FF**

Workaround: When an immediate read is required after a write to any register in the above address space, the workaround is to make two consecutive writes to the same address prior to the read. By this procedure, it ensures that the first write must complete and the read data will be correct.

This exception will not be fixed in future silicon revisions.

**Advisory
SYS_3***Impact on $I_{DDC(0)}$ Current if DSP Held in Reset Without Proper Initialization***Revision(s) Affected:** All Revisions**Details:** Due to the synchronous implementation of resets within the DSP subsystem, if the DSP is simply held in reset, it will not be in its lowest power state. The DSP must be properly initialized to enter its lowest power state.**Workaround:** The OMAP5910 DSP subsystem can go properly into a low current consumption state in any one of three different ways:

- Method 1: use the program in DSP PDRAM to put the DSP into idle
- Method 2: toggle the DSP reset (DSP_EN of ARM_RSTCT1): 0 then 1 then 0 again.
- Method 3: download a Program into the DSP that puts the DSP into idle

All of these must be done with DSP clocks enabled.

Method 1: Use the program code in DSP PDRAM to put the DSP into idle.

Enable DSP clock

Enable MPUI clock

Release DSP Interface Reset (Bit DSP_RST of ARM_RSTCT1)

Set up the MPUI

Set up the MPUI boot to the value 0x2 (API_DSPBootConfig(API_DSP_BOOT_IDLE))

Set API_SIZE_REGISTER register to 0 (Make SARAM inaccessible by MPU to allow the DSP to go into idle)

Release DSP reset (Bit DSP_EN of ARM_RSTCT1)

The DSP boots from the PDRAM and executes code that put itself in IDLE

Set all the IDLE bits of ARM_IDLECT1

Method 2: Toggle the DSP reset.

Cut DSP clock

Set all the IDLE bits of ARM_IDLECT1

Method 3: Download a Program into the DSP that puts the DSP into idle.

Turn on DSP clock

Turn on API clock

Release DSP Interface Reset (Bit DSP_RST of ARM_RSTCT1)

Set up the MPUI

Set up the MPUI boot to the value 0x5

(API_DSPBootConfig(API_DSP_BOOT_INTERNAL))

Write DSP code into DSP SARAM0

Set API_SIZE_REGISTER register to 0 (Make SARAM inaccessible by MPU to allow the DSP to go into IDLE)

Assert DSP reset (Bit DSP_EN of ARM_RSTCT1)

Release DSP reset (Bit DSP_EN of ARM_RSTCT1)

The DSP boots from SARAM0 and executes code that puts itself in idle

Set all the IDLE bits of ARM_IDLECT1

This exception will not be fixed in future silicon revisions.

Advisory
SYS_4

ESD Susceptibility

Revision(s) Affected: Revision G

Details: Revision G OMAP devices have pins that are susceptible to electrostatic discharge.

Workaround: None.

Special care should be taken for ESD prevention when handling these devices.

This exception will be fixed in future revisions.

8 Documentation Support

For device-specific data sheets and related documentation, visit the TI web site at: <http://www.ti.com>

For further information regarding the OMAP5910, please refer to:

- *OMAP5910 Dual-Core Processor Data Manual* (literature number SPRS197)
- *OMAP5910 Dual-Core Processor Technical Reference Manual* (literature number SPRU602)

For additional information, see the latest versions of:

- *TMS320C55x™ DSP Functional Overview* (literature number SPRU312)
- *TMS320C55x DSP CPU Reference Guide* (literature number SPRU371)
- *TMS320C55x DSP Mnemonic Instruction Set Reference Guide* (literature number SPRU374)
- *TMS320C55x DSP Algebraic Instruction Set Reference Guide* (literature number SPRU375)
- *TMS320C55x DSP CPU Programmer's Reference Supplement* (literature number SPRU652)

IMPORTANT NOTICE

Texas Instruments Incorporated and its subsidiaries (TI) reserve the right to make corrections, modifications, enhancements, improvements, and other changes to its products and services at any time and to discontinue any product or service without notice. Customers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. All products are sold subject to TI's terms and conditions of sale supplied at the time of order acknowledgment.

TI warrants performance of its hardware products to the specifications applicable at the time of sale in accordance with TI's standard warranty. Testing and other quality control techniques are used to the extent TI deems necessary to support this warranty. Except where mandated by government requirements, testing of all parameters of each product is not necessarily performed.

TI assumes no liability for applications assistance or customer product design. Customers are responsible for their products and applications using TI components. To minimize the risks associated with customer products and applications, customers should provide adequate design and operating safeguards.

TI does not warrant or represent that any license, either express or implied, is granted under any TI patent right, copyright, mask work right, or other TI intellectual property right relating to any combination, machine, or process in which TI products or services are used. Information published by TI regarding third-party products or services does not constitute a license from TI to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from TI under the patents or other intellectual property of TI.

Reproduction of information in TI data books or data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. Reproduction of this information with alteration is an unfair and deceptive business practice. TI is not responsible or liable for such altered documentation.

Resale of TI products or services with statements different from or beyond the parameters stated by TI for that product or service voids all express and any implied warranties for the associated TI product or service and is an unfair and deceptive business practice. TI is not responsible or liable for any such statements.

Mailing Address:

Texas Instruments
Post Office Box 655303
Dallas, Texas 75265